

# Metody Obliczeniowe w Nauce i Technice

## Układy równań liniowych - metody bezpośrednie

Adrianna Grobel

11.05.2026

### 1. Dane techniczne

- System operacyjny - macOS
- Procesor - Apple M3
- Pamięć RAM - 16 GB
- Język - Python 3.13.5
- Środowisko - PyCharm
- Precyzja obliczeń - typ float64 z biblioteki NumPy

### 2. Wprowadzenie

Celem laboratorium była implementacja metody eliminacji Gaussa do rozwiązywania układów równań liniowych  $Ax = b$  oraz zbadanie dokładności otrzymanych rozwiązań w zależności od uwarunkowania macierzy i użytej precyzji obliczeń.

W pierwszym etapie analizowano układ źle uwarunkowany, zadany macierzą  $A$  o wymiarach  $n \times n$  zdefiniowaną wzorem:

$$\begin{cases} a_{1j} = 1 \\ a_{ij} = \frac{1}{i+j-1} \end{cases} \quad \text{dla } i \neq 1 \quad \text{gdzie } i, j = 1, \dots, n$$

Powyższy układ będzie dalej określany jako układ (1).

Następnie eksperyment powtórzono dla układu dobrze uwarunkowanego, opartego na macierzy symetrycznej określonej wzorem:

$$\begin{cases} a_{ij} = \frac{2i}{j} & \text{dla } j \geq i \\ a_{ij} = a_{ji} & \text{dla } j < i \end{cases} \quad \text{gdzie } i, j = 1, \dots, n$$

Ten układ będzie dalej określany jako układ (2).

Ostatnim etapem ćwiczenia była implementacja zoptymalizowanego algorytmu Thomasa i zestawienie jego wydajności (czasu wykonania i zajętości pamięci) z klasyczną metodą Gaussa. Analizę przeprowadzono zadanej macierzy trójdzielnej:

$$\begin{cases} a_{i,i} = -3i - 7 \\ a_{i,i+1} = i \\ a_{i,i-1} = \frac{3}{i} & \text{dla } i > 1 \\ a_{i,j} = 0 & \text{dla } j < i - 1 \text{ oraz } j > i + 1 \end{cases} \quad \text{gdzie } i, j = 1, \dots, n$$

Układ zadany tą macierzą będzie dalej określany jako układ (3).

### 3. Realizacja ćwiczenia

Wszystkie eksperymenty numeryczne zaimplementowano w języku Python z wykorzystaniem biblioteki NumPy. Obliczenia prowadzono równolegle dla dwóch precyzji zmiennoprzecinkowych: pojedynczej (`np.float32`) i podwójnej (`np.float64`) oraz dla układów o różnych wymiarach  $n$ :

- $n \in \{2, 3, \dots, 20\}$  dla układu (1)
- $n \in \{2, 3, \dots, 20, 50, 100, 150\}$  dla układu (2)
- $n \in \{10, 50, 100, 250, 500, 1000\}$  dla układu (3)

Proces testowy rozpoczynano od wygenerowania referencyjnego wektora rozwiązania  $\bar{x}$  o elementach losowanych ze zbioru  $\{1, -1\}$  przy stałym ziarnie generatora liczb pseudolosowych (`np.random.seed(47654678)`) dla zapewnienia powtarzalności wyników. Następnie wyznaczano wektor wyrazów wolnych:

$$b = A\bar{x}$$

Układ  $Ax = b$  rozwiązywano traktując  $x$  jako niewiadomą, korzystając z eliminacji Gaussa z częściowym wyborem elementu wiodącego lub z algorytmu Thomasa. Wyznaczony w ten sposób wektor  $x$  porównywano następnie z wektorem referencyjnym  $\bar{x}$ .

#### 3.1. Zastosowane metody

##### Metoda eliminacji Gaussa

Metodę zastosowano dla układów (1) i (2). Aby zminimalizować błędy zaokrągleń maszynowych, przed redukcją kolumny algorytm zamieniał wiersze miejscami, wybierając element o największej wartości bezwzględnej, tzw. element wiodący  $a_{ii}$ . Następnie zerowano elementy pod przekątną według wzorów:

$$a_{jk} = a_{jk} - \left( \frac{a_{ji}}{a_{ii}} \right) \cdot a_{ik}, \quad b_j = b_j - \left( \frac{a_{ji}}{a_{ii}} \right) \cdot b_i$$

gdzie:  $j = i + 1, \dots, n$  oraz  $k = i, \dots, n$ , a  $i = 1, \dots, n - 1$  oznacza indeks aktualnie przetwarzanej kolumny

Wektor niewiadomych  $x$  wyznaczano od końca w procesie podstawiania wstecznego:

$$x_i = \frac{b_i - \sum_{j=i+1}^n a_{ij}x_j}{a_{ii}} \quad \text{dla } i = n, n-1, \dots, 1$$

Złożoność czasowa metody wynosi  $O(n^3)$ , a pamięciowa  $O(n^2)$ .

##### Algorytm Thomasa

Metodę zastosowano dla układu (3). Aby zoptymalizować obliczenia, zrezygnowano z operacji na elementach zerowych, a pełną macierz skompresowano do trzech wektorów: poddiagonalnego  $L$ , diagonalnego  $D$  oraz naddiagonalnego  $U$ . W pierwszej kolejności wyznaczano warunki brzegowe dla fazy eliminacji w przód:  $u'_1 = \frac{U_1}{D_1}$  oraz  $d'_1 = \frac{b_1}{D_1}$ . Następnie modyfikowano wektory według rekurencyjnych wzorów:

$$u'_i = \frac{U_i}{D_i - L_{i-1} \cdot u'_{i-1}}, \quad d'_i = \frac{b_i - L_{i-1} \cdot d'_{i-1}}{D_i - L_{i-1} \cdot u'_{i-1}}$$

gdzie:  $u'_i$  dla  $i = 2, \dots, n-1$  oraz  $d'_i$  dla  $i = 2, \dots, n$  – zmodyfikowane współczynniki nadprzekątnej oraz nowe wyrazy wolne, wyliczane w fazie eliminacji w przód,  $D_i, L_i, U_i$  – odpowiednio elementy głównej przekątnej, podprzekątnej oraz nadprzekątnej macierzy współczynników układu,  $b_i$  – kolejne elementy wektora wyrazów wolnych.

Wektor niewiadomych  $x$  wyznaczano od końca w uproszczonym procesie podstawiania wstecznego, przyjmując dla ostatniego elementu  $x_n = d'_n$ , a dla pozostałych stosując wzór:

$$x_i = d'_i - u'_i \cdot x_{i+1} \quad \text{dla } i = n-1, n-2, \dots, 1$$

### 3.2. Metryka błędu

Dokładność rozwiązań oceniano na podstawie błędu bezwzględnego w normie maksimum:

$$\text{Błąd} = \|x - \bar{x}\|_{\max} = \max_{i \in \{1, \dots, n\}} |x_i - \bar{x}_i|$$

gdzie:  $x$  – wektor wyznaczony numerycznie,  $\bar{x}$  – wektor referencyjny (prawdziwy),  $n$  – liczba elementów wektora.

### 3.3. Uwarunkowanie macierzy

Wskaźnik uwarunkowania wyznaczano w celu oceny wrażliwości układów na błędy numeryczne. Dany jest on wzorem:

$$\text{cond}(A) = \|A\| \cdot \|A^{-1}\|$$

gdzie  $\|\cdot\|$  oznacza normę operatorową, zdefiniowaną jako maksymalna suma wartości bezwzględnych elementów w kolumnie macierzy:

$$\|A\|_1 = \max_{1 \leq j \leq n} \sum_{i=1}^n |a_{ij}|$$

W obliczeniach wykorzystano funkcję `numpy.linalg.cond`. Pozwoliło to uniknąć błędów zaokrągleń, które wystąpiłyby przy próbie bezpośredniego odwracania macierzy.

## 4. Wyniki

### 4.1. Analiza układu (1)

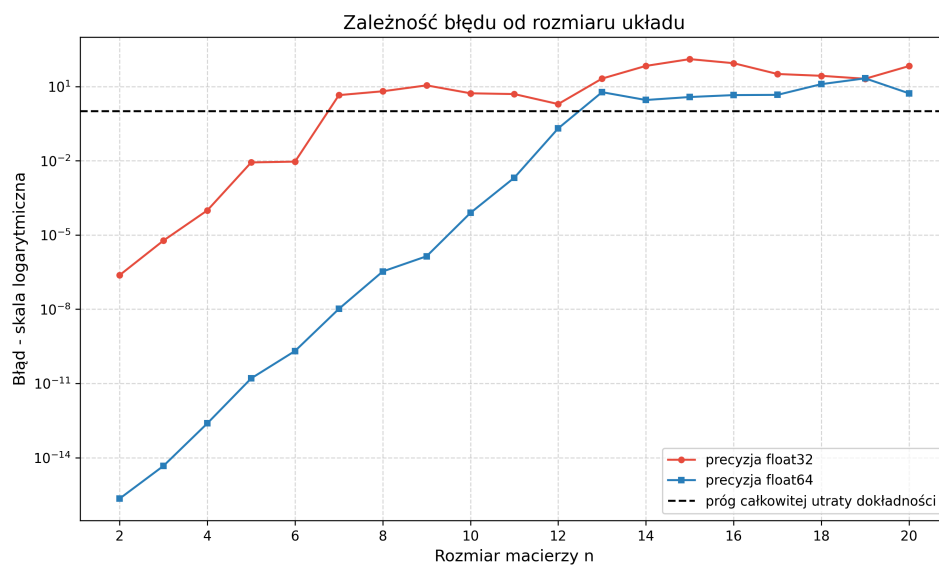
W Tabeli 1 zestawiono wartości współczynnika uwarunkowania macierzy dla obu badanych precyzji zmiennoprzecinkowych oraz obliczony błąd bezwzględny rozwiązania w zależności od rozmiaru układu  $n$ .

| n  | cond(A)              |                      | Błąd                 |                       |
|----|----------------------|----------------------|----------------------|-----------------------|
|    | float32              | float64              | float32              | float64               |
| 2  | $1.41 \cdot 10^1$    | $1.41 \cdot 10^1$    | $2.38 \cdot 10^{-7}$ | $2.22 \cdot 10^{-16}$ |
| 3  | $4.83 \cdot 10^2$    | $4.83 \cdot 10^2$    | $5.96 \cdot 10^{-6}$ | $4.66 \cdot 10^{-15}$ |
| 4  | $1.70 \cdot 10^4$    | $1.70 \cdot 10^4$    | $9.88 \cdot 10^{-5}$ | $2.44 \cdot 10^{-13}$ |
| 5  | $5.92 \cdot 10^5$    | $5.92 \cdot 10^5$    | $8.57 \cdot 10^{-3}$ | $1.61 \cdot 10^{-11}$ |
| 6  | $2.10 \cdot 10^7$    | $2.04 \cdot 10^7$    | $9.24 \cdot 10^{-3}$ | $2.05 \cdot 10^{-10}$ |
| 7  | $1.33 \cdot 10^9$    | $6.98 \cdot 10^8$    | $4.49 \cdot 10^0$    | $1.03 \cdot 10^{-8}$  |
| 8  | $3.03 \cdot 10^9$    | $2.38 \cdot 10^{10}$ | $6.43 \cdot 10^0$    | $3.36 \cdot 10^{-7}$  |
| 9  | $8.66 \cdot 10^9$    | $8.13 \cdot 10^{11}$ | $1.12 \cdot 10^1$    | $1.38 \cdot 10^{-6}$  |
| 10 | $3.63 \cdot 10^9$    | $2.77 \cdot 10^{13}$ | $5.33 \cdot 10^0$    | $7.83 \cdot 10^{-5}$  |
| 11 | $1.96 \cdot 10^9$    | $9.43 \cdot 10^{14}$ | $4.96 \cdot 10^0$    | $2.07 \cdot 10^{-3}$  |
| 12 | $2.36 \cdot 10^9$    | $3.25 \cdot 10^{16}$ | $1.96 \cdot 10^0$    | $2.03 \cdot 10^{-1}$  |
| 13 | $3.73 \cdot 10^9$    | $7.73 \cdot 10^{17}$ | $2.09 \cdot 10^1$    | $5.91 \cdot 10^0$     |
| 14 | $3.09 \cdot 10^9$    | $1.36 \cdot 10^{18}$ | $6.81 \cdot 10^1$    | $2.86 \cdot 10^0$     |
| 15 | $7.63 \cdot 10^9$    | $9.24 \cdot 10^{17}$ | $1.28 \cdot 10^2$    | $3.77 \cdot 10^0$     |
| 16 | $1.15 \cdot 10^{10}$ | $1.50 \cdot 10^{18}$ | $8.77 \cdot 10^1$    | $4.51 \cdot 10^0$     |
| 17 | $1.67 \cdot 10^{10}$ | $7.56 \cdot 10^{18}$ | $3.21 \cdot 10^1$    | $4.60 \cdot 10^0$     |
| 18 | $1.22 \cdot 10^{10}$ | $1.95 \cdot 10^{18}$ | $2.71 \cdot 10^1$    | $1.25 \cdot 10^1$     |
| 19 | $1.21 \cdot 10^{10}$ | $6.88 \cdot 10^{18}$ | $2.06 \cdot 10^1$    | $2.14 \cdot 10^1$     |
| 20 | $2.31 \cdot 10^{10}$ | $2.69 \cdot 10^{18}$ | $6.71 \cdot 10^1$    | $5.31 \cdot 10^0$     |

Tabela 1: Zestawienie uwarunkowania macierzy oraz błędów rozwiązania dla układu (1).

Z analizy Tabela 1 wynika, że badany układ jest skrajnie źle uwarunkowany, co oznacza, że jest wysoce podatny na potęgowanie błędów zaokrągleń maszynowych, które zniekształcają wynik. Wraz ze wzrostem wymiaru  $n$  następuje wykładniczy wzrost błędu rozwiązania, co jest bezpośrednio powiązane z gigantycznym przyrostem wartości współczynnika  $\text{cond}(A)$ . Rozbieżność w wynikach uwarunkowania wynika z obliczania go przez funkcję `numpy.linalg.cond` jako stosunku największej do najmniejszej wartości osobliwej macierzy:  $\frac{\sigma_{\max}}{\sigma_{\min}}$ . Dla dużych układów wartość  $\sigma_{\min}$  dąży do zera i ginie w szumie numerycznym precyzji float32, której epsilon maszynowy wynosi około  $10^{-7}$ . Uniemożliwia to poprawne dzielenie i sztucznie zatrzymuje wynik na rzędzie  $10^{10}$ . Dopiero typ float64 ze znacznie mniejszym epsilon na poziomie  $10^{-16}$  wychwytuje te mikroskopijne wartości mianownika, ujawniając faktyczny wzrost uwarunkowania aż do rzędu  $10^{18}$ . Zgodnie z przewidywaniami, błąd generowany w precyzji pojedynczej rośnie znacznie szybciej niż w podwójnej, co skutkuje wcześniejszym załamaniem algorytmu – dla float32 następuje to już przy  $n = 7$ , gdzie błąd nagle skacze do poziomu 4.49. Zastosowanie podwójnej precyzji (float64) pozwala opóźnić ten efekt do  $n = 13$ , generując błąd na poziomie 5.91. Powyżej wspomnianych wymiarów załamania, błąd przestaje rosnać w sposób uporządkowany i zaczyna oscylować w losowy sposób wokół dużych wartości. Omawiany trend utraty dokładności zilustrowano w skali logarytmicznej na Rysunek 1,

gdzie przerywaną linią zaznaczono poziom błędu, oznaczający moment całkowitej utraty dokładności rozwiązania.



Rysunek 1: Zależność błędu w skali logarytmicznej od rozmiaru macierzy  $n$  dla układu (1).

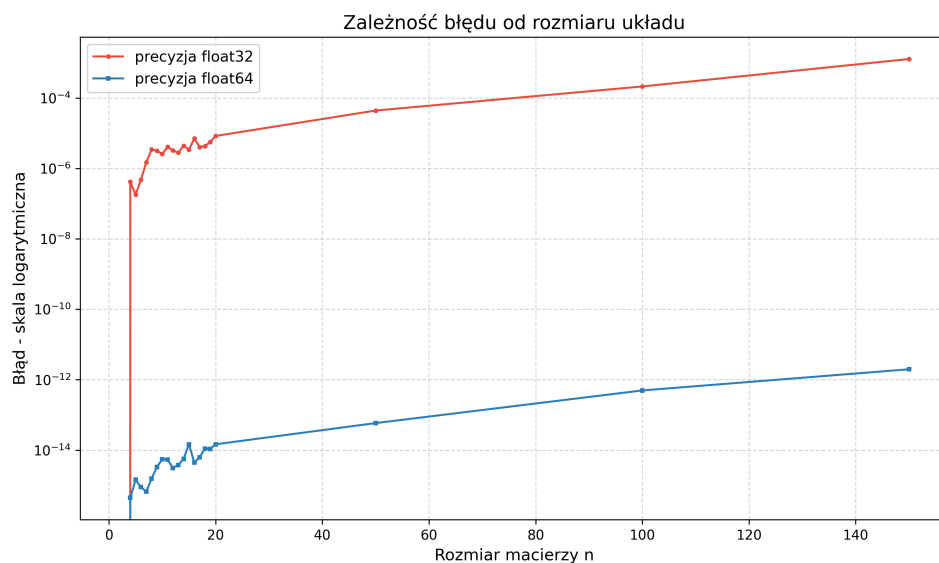
## 4.2. Analiza układu (2)

W Tabeli 2 zestawiono wartości współczynnika uwarunkowania macierzy dla obu badanych precyzji zmiennoprzecinkowych oraz obliczony błąd bezwzględny rozwiązania w zależności od rozmiaru układu  $n$ .

| n   | cond(A)           |                   | Błąd                 |                       |
|-----|-------------------|-------------------|----------------------|-----------------------|
|     | float32           | float64           | float32              | float64               |
| 2   | $3.00 \cdot 10^0$ | $3.00 \cdot 10^0$ | $0.00 \cdot 10^0$    | $0.00 \cdot 10^0$     |
| 3   | $6.66 \cdot 10^0$ | $6.66 \cdot 10^0$ | $0.00 \cdot 10^0$    | $0.00 \cdot 10^0$     |
| 4   | $1.22 \cdot 10^1$ | $1.22 \cdot 10^1$ | $4.17 \cdot 10^{-7}$ | $4.44 \cdot 10^{-16}$ |
| 5   | $1.97 \cdot 10^1$ | $1.97 \cdot 10^1$ | $1.79 \cdot 10^{-7}$ | $1.44 \cdot 10^{-15}$ |
| 6   | $2.90 \cdot 10^1$ | $2.90 \cdot 10^1$ | $4.77 \cdot 10^{-7}$ | $8.88 \cdot 10^{-16}$ |
| 7   | $4.04 \cdot 10^1$ | $4.04 \cdot 10^1$ | $1.49 \cdot 10^{-6}$ | $6.66 \cdot 10^{-16}$ |
| 8   | $5.37 \cdot 10^1$ | $5.37 \cdot 10^1$ | $3.46 \cdot 10^{-6}$ | $1.55 \cdot 10^{-15}$ |
| 9   | $6.90 \cdot 10^1$ | $6.90 \cdot 10^1$ | $3.16 \cdot 10^{-6}$ | $3.33 \cdot 10^{-15}$ |
| 10  | $8.64 \cdot 10^1$ | $8.64 \cdot 10^1$ | $2.56 \cdot 10^{-6}$ | $5.55 \cdot 10^{-15}$ |
| 11  | $1.06 \cdot 10^2$ | $1.06 \cdot 10^2$ | $4.11 \cdot 10^{-6}$ | $5.33 \cdot 10^{-15}$ |
| 12  | $1.27 \cdot 10^2$ | $1.27 \cdot 10^2$ | $3.22 \cdot 10^{-6}$ | $3.11 \cdot 10^{-15}$ |
| 13  | $1.51 \cdot 10^2$ | $1.51 \cdot 10^2$ | $2.80 \cdot 10^{-6}$ | $3.77 \cdot 10^{-15}$ |
| 14  | $1.76 \cdot 10^2$ | $1.76 \cdot 10^2$ | $4.41 \cdot 10^{-6}$ | $5.66 \cdot 10^{-15}$ |
| 15  | $2.04 \cdot 10^2$ | $2.04 \cdot 10^2$ | $3.40 \cdot 10^{-6}$ | $1.47 \cdot 10^{-14}$ |
| 16  | $2.33 \cdot 10^2$ | $2.33 \cdot 10^2$ | $7.03 \cdot 10^{-6}$ | $4.44 \cdot 10^{-15}$ |
| 17  | $2.65 \cdot 10^2$ | $2.65 \cdot 10^2$ | $4.05 \cdot 10^{-6}$ | $6.22 \cdot 10^{-15}$ |
| 18  | $2.99 \cdot 10^2$ | $2.99 \cdot 10^2$ | $4.29 \cdot 10^{-6}$ | $1.11 \cdot 10^{-14}$ |
| 19  | $3.35 \cdot 10^2$ | $3.35 \cdot 10^2$ | $5.66 \cdot 10^{-6}$ | $1.09 \cdot 10^{-14}$ |
| 20  | $3.72 \cdot 10^2$ | $3.72 \cdot 10^2$ | $8.34 \cdot 10^{-6}$ | $1.47 \cdot 10^{-14}$ |
| 50  | $2.49 \cdot 10^3$ | $2.49 \cdot 10^3$ | $4.40 \cdot 10^{-5}$ | $5.86 \cdot 10^{-14}$ |
| 100 | $1.03 \cdot 10^4$ | $1.03 \cdot 10^4$ | $2.12 \cdot 10^{-4}$ | $4.96 \cdot 10^{-13}$ |
| 150 | $2.34 \cdot 10^4$ | $2.34 \cdot 10^4$ | $1.28 \cdot 10^{-3}$ | $1.98 \cdot 10^{-12}$ |

Tabela 2: Zestawienie uwarunkowania macierzy oraz błędów rozwiązywania dla układu (2).

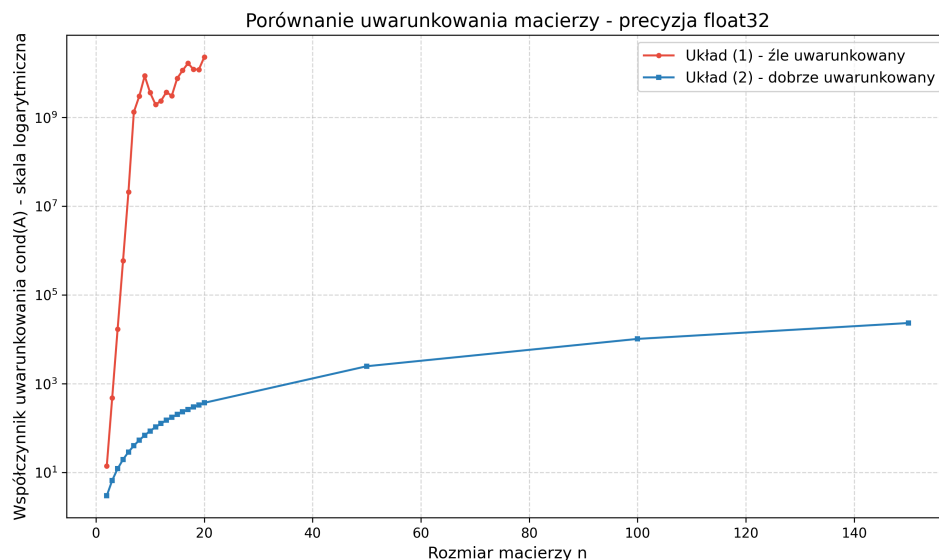
Z analizy Tabela 2 wynika, że układ (2) jest dobrze uwarunkowany i stabilny numerycznie. Wraz ze wzrostem rozmiaru macierzy  $n$  następuje powolny wzrost zarówno błędu rozwiązania, jak i uwarunkowania macierzy, który jest identyczny dla obu precyzji i osiąga maksymalnie rząd  $10^4$ . Błąd dla precyzji pojedynczej (float32) jest znacznie większy i rośnie szybciej, osiągając ostatecznie wartości rzędu  $10^{-3}$ , podczas gdy zastosowanie precyzji podwójnej (float64) utrzymuje błąd na poziomie  $10^{-12}$ . Układ ten w żadnym momencie nie ulega załamaniu numerycznemu. Omawianą zależność błędu od rozmiaru macierzy zilustrowano w skali logarytmicznej na Rysunek 2.



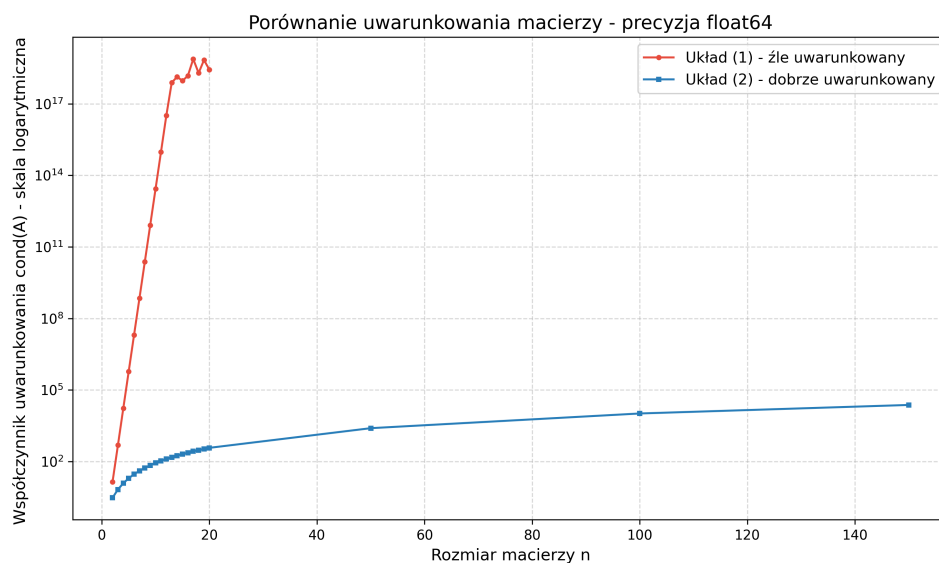
Rysunek 2: Zależność błędu w skali logarytmicznej od rozmiaru macierzy  $n$  dla układu (2).

### 4.3. Zestawienie współczynników uwarunkowania

Zestawienie współczynników uwarunkowania na Rysunek 3 i Rysunek 4 bezpośrednio tłumaczy różnice w dokładności obu układów. W skali logarytmicznej wyraźnie widać, że  $\text{cond}(A)$  układu (1) rośnie wykładniczo, błyskawicznie osiągając wartości krytyczne dla stabilności numerycznej. Z kolei układ (2) wykazuje bardzo łagodny wzrost uwarunkowania, co gwarantuje zachowanie wysokiej precyzji.



Rysunek 3: Porównanie współczynnika uwarunkowania dla układów (1) i (2) dla float32.



Rysunek 4: Porównanie współczynnika uwarunkowania dla układów (1) i (2) dla float64.

#### 4.4. Porównanie algorytmu Gaussa i algorytmu Thomasa

W Tabeli 3 i Tabeli 4 zestawiono wyniki obliczeń dla układu o macierzy trójdzielnej (3) uzyskane za pomocą algorytmu eliminacji Gaussa oraz algorytmu Thomasa. Dla każdego rozmiaru  $n$  wyznaczono wskaźnik uwarunkowania macierzy oraz błąd rozwiązania. Czas wykonania obu metod, z pominięciem budowy układu, zmierzono przy użyciu licznika `time.perf_counter()`. Całkowite zużycie pamięci w bajtach wyznaczono natomiast poprzez zsumowanie wielkości zaalokowanych struktur biblioteki NumPy za pomocą atrybutu `.nbytes`. Badania przeprowadzono dla pojedynczej (float32) oraz podwójnej (float64) precyzji obliczeniowej.

| n    | cond(A)           | Eliminacja Gaussa    |                      |            | Algorytm Thomasa     |                      |            |
|------|-------------------|----------------------|----------------------|------------|----------------------|----------------------|------------|
|      |                   | Błąd                 | Czas [s]             | Pamięć [B] | Błąd                 | Czas [s]             | Pamięć [B] |
| 10   | $1.34 \cdot 10^0$ | $1.19 \cdot 10^{-7}$ | $1.24 \cdot 10^{-4}$ | 440        | $1.19 \cdot 10^{-7}$ | $8.67 \cdot 10^{-6}$ | 152        |
| 20   | $1.34 \cdot 10^0$ | $1.19 \cdot 10^{-7}$ | $2.94 \cdot 10^{-4}$ | 1 680      | $1.19 \cdot 10^{-7}$ | $1.42 \cdot 10^{-5}$ | 312        |
| 30   | $1.34 \cdot 10^0$ | $1.19 \cdot 10^{-7}$ | $5.49 \cdot 10^{-4}$ | 3 720      | $1.19 \cdot 10^{-7}$ | $1.87 \cdot 10^{-5}$ | 472        |
| 40   | $1.34 \cdot 10^0$ | $1.19 \cdot 10^{-7}$ | $9.50 \cdot 10^{-4}$ | 6 560      | $1.19 \cdot 10^{-7}$ | $2.41 \cdot 10^{-5}$ | 632        |
| 50   | $1.34 \cdot 10^0$ | $1.19 \cdot 10^{-7}$ | $1.48 \cdot 10^{-3}$ | 10 200     | $1.19 \cdot 10^{-7}$ | $3.21 \cdot 10^{-5}$ | 792        |
| 60   | $1.34 \cdot 10^0$ | $1.19 \cdot 10^{-7}$ | $2.12 \cdot 10^{-3}$ | 14 640     | $1.19 \cdot 10^{-7}$ | $3.55 \cdot 10^{-5}$ | 952        |
| 70   | $1.34 \cdot 10^0$ | $1.19 \cdot 10^{-7}$ | $2.97 \cdot 10^{-3}$ | 19 880     | $1.19 \cdot 10^{-7}$ | $4.09 \cdot 10^{-5}$ | 1 112      |
| 80   | $1.34 \cdot 10^0$ | $1.19 \cdot 10^{-7}$ | $3.89 \cdot 10^{-3}$ | 25 920     | $1.19 \cdot 10^{-7}$ | $5.07 \cdot 10^{-5}$ | 1 272      |
| 90   | $1.34 \cdot 10^0$ | $1.19 \cdot 10^{-7}$ | $4.83 \cdot 10^{-3}$ | 32 760     | $1.19 \cdot 10^{-7}$ | $5.61 \cdot 10^{-5}$ | 1 432      |
| 100  | $1.34 \cdot 10^0$ | $1.19 \cdot 10^{-7}$ | $6.07 \cdot 10^{-3}$ | 40 400     | $1.19 \cdot 10^{-7}$ | $6.42 \cdot 10^{-5}$ | 1 592      |
| 250  | $1.34 \cdot 10^0$ | $1.19 \cdot 10^{-7}$ | $3.68 \cdot 10^{-2}$ | 251 000    | $1.19 \cdot 10^{-7}$ | $1.63 \cdot 10^{-4}$ | 3 992      |
| 500  | $1.34 \cdot 10^0$ | $1.19 \cdot 10^{-7}$ | $1.52 \cdot 10^{-1}$ | 1 002 000  | $1.19 \cdot 10^{-7}$ | $3.18 \cdot 10^{-4}$ | 7 992      |
| 1000 | $1.34 \cdot 10^0$ | $1.19 \cdot 10^{-7}$ | $6.58 \cdot 10^{-1}$ | 4 004 000  | $1.19 \cdot 10^{-7}$ | $6.40 \cdot 10^{-4}$ | 15 992     |

Tabela 3: Zestawienie uwarunkowania, błędów, czasów wykonania i pamięci dla układu (3) – pojedyncza precyzja (float32).

| n    | cond(A)           | Eliminacja Gaussa     |                      |            | Algorytm Thomasa      |                      |            |
|------|-------------------|-----------------------|----------------------|------------|-----------------------|----------------------|------------|
|      |                   | Błąd                  | Czas [s]             | Pamięć [B] | Błąd                  | Czas [s]             | Pamięć [B] |
| 10   | $1.34 \cdot 10^0$ | $2.22 \cdot 10^{-16}$ | $1.02 \cdot 10^{-4}$ | 880        | $2.22 \cdot 10^{-16}$ | $8.08 \cdot 10^{-6}$ | 304        |
| 20   | $1.34 \cdot 10^0$ | $2.22 \cdot 10^{-16}$ | $2.81 \cdot 10^{-4}$ | 3 360      | $2.22 \cdot 10^{-16}$ | $1.24 \cdot 10^{-5}$ | 624        |
| 30   | $1.34 \cdot 10^0$ | $2.22 \cdot 10^{-16}$ | $5.81 \cdot 10^{-4}$ | 7 440      | $2.22 \cdot 10^{-16}$ | $1.95 \cdot 10^{-5}$ | 944        |
| 40   | $1.34 \cdot 10^0$ | $2.22 \cdot 10^{-16}$ | $9.54 \cdot 10^{-4}$ | 13 120     | $2.22 \cdot 10^{-16}$ | $2.37 \cdot 10^{-5}$ | 1 264      |
| 50   | $1.34 \cdot 10^0$ | $2.22 \cdot 10^{-16}$ | $1.52 \cdot 10^{-3}$ | 20 400     | $2.22 \cdot 10^{-16}$ | $2.95 \cdot 10^{-5}$ | 1 584      |
| 60   | $1.34 \cdot 10^0$ | $2.22 \cdot 10^{-16}$ | $2.12 \cdot 10^{-3}$ | 29 280     | $2.22 \cdot 10^{-16}$ | $3.89 \cdot 10^{-5}$ | 1 904      |
| 70   | $1.34 \cdot 10^0$ | $2.22 \cdot 10^{-16}$ | $2.96 \cdot 10^{-3}$ | 39 760     | $2.22 \cdot 10^{-16}$ | $4.35 \cdot 10^{-5}$ | 2 224      |
| 80   | $1.34 \cdot 10^0$ | $2.22 \cdot 10^{-16}$ | $3.81 \cdot 10^{-3}$ | 51 840     | $2.22 \cdot 10^{-16}$ | $5.02 \cdot 10^{-5}$ | 2 544      |
| 90   | $1.34 \cdot 10^0$ | $2.22 \cdot 10^{-16}$ | $4.77 \cdot 10^{-3}$ | 65 520     | $2.22 \cdot 10^{-16}$ | $5.56 \cdot 10^{-5}$ | 2 864      |
| 100  | $1.34 \cdot 10^0$ | $2.22 \cdot 10^{-16}$ | $5.99 \cdot 10^{-3}$ | 80 800     | $2.22 \cdot 10^{-16}$ | $6.21 \cdot 10^{-5}$ | 3 184      |
| 250  | $1.34 \cdot 10^0$ | $2.22 \cdot 10^{-16}$ | $3.77 \cdot 10^{-2}$ | 502 000    | $2.22 \cdot 10^{-16}$ | $1.54 \cdot 10^{-4}$ | 7 984      |
| 500  | $1.34 \cdot 10^0$ | $2.22 \cdot 10^{-16}$ | $1.61 \cdot 10^{-1}$ | 2 004 000  | $2.22 \cdot 10^{-16}$ | $3.21 \cdot 10^{-4}$ | 15 984     |
| 1000 | $1.34 \cdot 10^0$ | $2.22 \cdot 10^{-16}$ | $7.21 \cdot 10^{-1}$ | 8 008 000  | $2.22 \cdot 10^{-16}$ | $6.08 \cdot 10^{-4}$ | 31 984     |

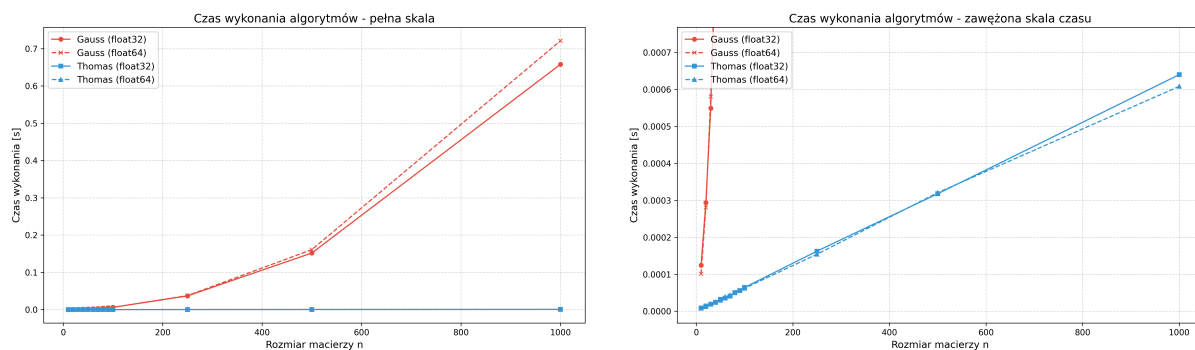
Tabela 4: Zestawienie uwarunkowania, błędów, czasów wykonania i pamięci dla układu (3) – podwójna precyzja (float64).

Z analizy danych zawartych w Tabeli 3 i Tabeli 4 wynika, że obie metody generują identyczne błędy bezwzględne, które pozostają na stałym, minimalnym poziomie niezależnie od wymiaru układu  $n$ . Dokładność obliczeń zależy wyłącznie od wybranej precyzji: wynosi ona ok.  $1.19 \cdot 10^{-7}$  dla float32 oraz ok.  $2.22 \cdot 10^{-16}$  dla float64, co potwierdza świetne uwarunkowanie macierzy trójdzielnej –  $\text{cond}(A) \approx 1.34$ .

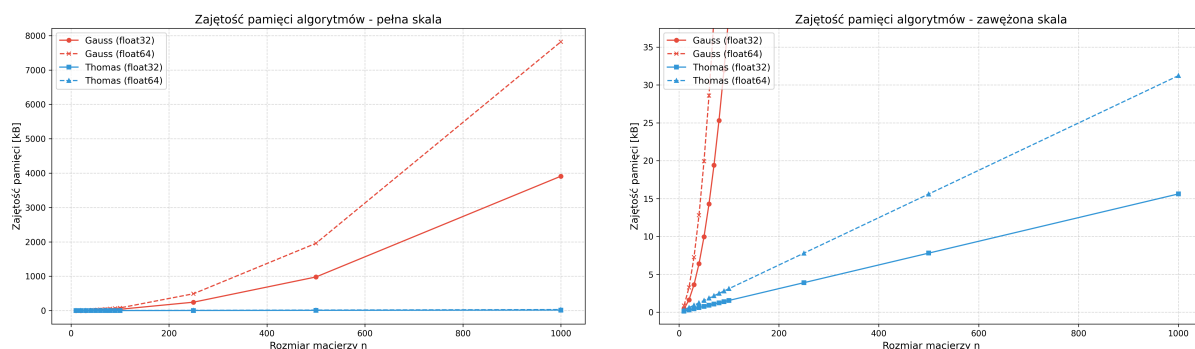
Zasadnicza różnica dotyczy wydajności algorytmów. Wraz ze wzrostem  $n$ , czas wykonania eliminacji Gaussa drastycznie rośnie – dla  $n = 1000$  obliczenia trwają około 1 s. Algorytm Thomasa rozwiązuje ten sam układ w ułamku milisekundy, ok.  $6 \cdot 10^{-4}$  s. Zastosowanie wyższej precyzji, float64 zamiast float32 w obu algorytmach wydłuża czas wykonania jedynie w sposób marginalny.

Przepaść zaobserwowano również w zajętości pamięci: dla największego badanego układu pełna macierz stosowana w metodzie Gaussa wymaga około 4 MB (dla f32) lub 8 MB (dla f64), podczas gdy algorytm Thomasa potrzebuje jedynie odpowiednio ok. 16 KB lub 32 KB. Zwiększenie precyzji zawsze skutkuje dokładnie dwukrotnym wzrostem zapotrzebowania na pamięć. Otrzymane wyniki stanowią doskonałe, praktyczne potwierdzenie redukcji złożoności obliczeniowej z  $O(n^3)$  do  $O(n)$  oraz złożoności pamięciowej z  $O(n^2)$  do  $O(n)$  dla algorytmu Thomasa.

Na Rysunek 5 zestawiono wydajność czasową, natomiast na Rysunek 6 zapotrzebowanie pamięciowe obu algorytmów. Dla każdego układu zaprezentowano widok w pełnej skali, ukazujący drastyczny przyrost danej metryki dla Gaussa oraz widok z zawężoną skalą czasu, pozwalający dostrzec liniowy charakter algorytmu Thomasa.



Rysunek 5: Czas wykonania układu (3). Pełna skala po stronie lewej, a zawężona po prawej.



Rysunek 6: Zużycie pamięci układu (3). Pełna skala po stronie lewej, a zawężona po prawej.

## 5. Podsumowanie

Zrealizowane ćwiczenie udowodniło, że uwarunkowanie macierzy jest kluczowe dla stabilności obliczeń numerycznych – układy źle uwarunkowane nieuchronnie ulegają załamaniu numerycznemu podczas zwiększania  $n$ , podczas gdy dobrze uwarunkowane zapewniają wysoką dokładność niezależnie od swojego wymiaru. Zastosowanie podwójnej precyzji (float64 zamiast float32) skutecznie ogranicza błędy zaokrągleń jedynie w początkowej fazie wzrostu rozmiaru układu, przy pomijalnym narzucie czasowym i dwukrotnie większym zużyciu pamięci, po czym błędy numeryczne i tak ulegają kumulacji. Z kolei z perspektywy wydajności, zastąpienie metody eliminacji Gaussa algorytmem Thomasa dla macierzy trójdzielnych obniżyło teoretyczną złożoność obliczeniową z rzędu  $O(n^3)$  do liniowej  $O(n)$ , a złożoność pamięciową z  $O(n^2)$  do  $O(n)$ .